# Phase 5.4
# STM32 Takes Over
# LTE Auto-Registration

Submission 1: STM32 firmware connects to the relay over 4G with no human intervention

**STATUS: COMPLETED on 2026-04-29.**

STM32 firmware boots, automatically powers up the LTE network connection, opens TCP through ngrok to the relay server, registers its phone number, and waits in idle. No manual AT commands. The relay log on the laptop confirms: + registered 01000000001.

This document captures the working hardware configuration, the firmware code, the two Arduino sketches that prepare the LilyGO board, and every gotcha that had to be solved along the way.

# 1. Goal of Phase 5.4

In Phase 5.3 the human was typing AT commands into Arduino Serial Monitor to register the device with the relay. Phase 5.4 puts the STM32 in charge of doing the same thing automatically.

```
Phase 5.3 (manual):
       Human  ■■■■■  Arduino Serial Monitor  ■■■■■  ESP32 bridge  ■■■■■  A7608 modem
                     (typing AT commands)

   Phase 5.4 (automated):
       STM32 firmware  ■■■■■  USART1  ■■■■■  A7608 modem  ■■■■■  4G  ■■■■■  ngrok  ■■■■■  relay.py
       (sends AT sequence
        and REG automatically
        on every boot)
```

End result: power on the device, wait ~25 seconds, and it's registered with the relay and ready to receive incoming calls. Status LED slow-blinks at 1 Hz when registered.

# 2. Architecture

```
STM32 Black Pill                          LilyGO XY-A7608B v1.2
   ■■■■■■■■■■■■■■■■■■                          ■■■■■■■■■■■■■■■■■■■■■■■■■
                                              ■■■■■■■■■■■■■■■■■■■■■■■■■
   PA8 ■■■■■ ESP32_RST ■■■■■■■■HOLD LOW■■■■■■■■■■■■■ ESP32 (in deep sleep■
                                              ■  after one-time      ■
                                              ■  power-on sketch)    ■
                                              ■■■■■■■■■■■■■■■■■■■■■■■■■

   PB1 ■■■■■ MODEM_POWERON ■■■■■■■HIGH■■■■■■■■■■■■■ A7608 power supply enable
   PB2 ■■■■■ MODEM_PWRKEY  ■■■■■■(idle HIGH)■■■■■■ A7608 virtual power button

   PA9  (USART1 TX, 115200) ■■■■■■■■■data■■■■■■■■ A7608 RXD (LilyGO IO26 / TX label)
   PA10 (USART1 RX, 115200) ■■■■■■■■data■■■■■■■■ A7608 TXD (LilyGO IO27 / RX label)
   GND  ■■■■■■■■■■■■■■ shared ground ■■■■■■■■■■■■■ GND

   Status LED: PC13 (onboard, active LOW)
   Buttons:    PA1 = Call,  PB0 = Answer/Hangup  (input pull-up to GND)
```

The ESP32 on the LilyGO board does the modem's hardware power-on sequence ONCE (via a small Arduino sketch flashed to it), then enters deep sleep. From that point on, the ESP32 is electrically silent and the STM32 has clean access to the modem's UART through GPIO pins 26 and 27.

# 3. Hardware Setup

## 3.1 — Components per device

| Component | Purpose |
|---|---|
| STM32F401RCT6 Black Pill | Microcontroller running the application firmware |
| LilyGO XY-A7608B v1.2 | LTE modem board (ESP32 + A7608 SIMCom 4G modem) |
| ST-LINK V2 | For programming + debugging the STM32 |
| Vodafone Egypt SIM | Cellular subscription with active data plan |
| Push-buttons (×2) | Call (PA1) and Answer/Hangup (PB0) |
| USB-C cable to PC | Powers the STM32 and is the SWD link via ST-Link |
| USB-C cable to phone charger | Powers the LilyGO board separately |
| Jumper wires | Six wires between STM32 and LilyGO (+ button wires) |

## 3.2 — The 6 wires between STM32 and LilyGO

| # | STM32 pin | LilyGO pin | Function |
|---|---|---|---|
| 1 | PA8 (output, LOW) | RST (right header) | Holds ESP32 in reset (defensive — ESP32 is also asleep from sketch) |
| 2 | PB1 (output, HIGH) | IO12 (left header) | Modem POWERON enable — keeps the A7608 chip powered |
| 3 | PB2 (output, HIGH idle) | IO4 (left header) | Modem PWRKEY — pulsed by ESP32 sketch at boot, then idle HIGH |
| 4 | PA9 (USART1 TX) | IO26 / "TX" label | Data: STM32 → modem |
| 5 | PA10 (USART1 RX) | IO27 / "RX" label | Data: modem → STM32 |
| 6 | GND | GND | Common ground (mandatory for any UART) |

## 3.3 — Buttons (2 wires + GND each)

| # | STM32 pin | Other side |
|---|---|---|
| 7 | PA1 (input, pull-up) | Call button → GND rail |
| 8 | PB0 (input, pull-up) | Answer/Hangup button → GND rail |

## 3.4 — Power

**Each board has its own power source.** They share only one wire: ground.

| Board | Powered by |
|---|---|
| STM32 Black Pill | USB-C cable to PC (also used for SWD via ST-Link) |
| LilyGO LTE board | USB-C cable to phone charger (5 V/2 A) OR 18650 Li-ion battery |

Critical: The LilyGO board CANNOT be powered from the STM32. The 4G modem can draw 2 A bursts during transmit, while the STM32's regulator only provides 300 mA.

# 4. Complete STM32 Pin Map (after Phase 5.4)

| Pin | Function | Connection |
|---|---|---|
| PA0 | ADC1_IN0 | MAX9814 OUT (audio — disabled in 5.4) |
| PA1 | GPIO input pull-up | Call button → GND |
| PA8 | GPIO output (LOW) | LilyGO RST (ESP32 reset hold) |
| PA9 | USART1 TX | LilyGO IO26 (modem RX line) |
| PA10 | USART1 RX | LilyGO IO27 (modem TX line) |
| PA13 | SWDIO | ST-Link |
| PA14 | SWCLK | ST-Link |
| PB0 | GPIO input pull-up | Answer/Hangup button → GND |
| PB1 | GPIO output (HIGH) | LilyGO IO12 (POWERON) |
| PB2 | GPIO output (HIGH idle) | LilyGO IO4 (PWRKEY) |
| PB6 | I2C1_SCL | MCP4725 SCL (audio — disabled in 5.4) |
| PB7 | I2C1_SDA | MCP4725 SDA (audio — disabled in 5.4) |
| PC13 | GPIO output | Onboard status LED |

# 5. Two Arduino Sketches Used Once on the LilyGO

Both sketches are uploaded to the ESP32 chip on the LilyGO board (not to the STM32). The first locks the modem baud permanently. The second hands control to the STM32. After both are run, you never need Arduino IDE again.

## 5.1 — Factory Reset Utility (lock baud at 115200 forever)

Many SIMCom A7608 modems ship with auto-baud or a non-115200 default. The STM32 firmware expects 115200, so we lock the modem permanently using BOTH AT+IPR=115200 (current session) AND AT+IPREX=115200 (boot baud, non-volatile).

```
#include <HardwareSerial.h>

HardwareSerial gsmSerial(2);

#define MODEM_RX_PIN  27
#define MODEM_TX_PIN  26
#define MODEM_PWRKEY_PIN  4
#define MODEM_POWERON_PIN 12
#define MODEM_RST_PIN  5
#define PC_BAUD        115200

const long BAUDS[]  = {115200, 9600, 921600, 460800, 230400, 57600, 38400, 19200};
const int  NUM_BAUDS = 8;

bool sendAndWaitForOK(const char* cmd, unsigned long timeout_ms) {
    while (gsmSerial.available()) gsmSerial.read();
    gsmSerial.print(cmd); gsmSerial.print("\r\n");
    String resp;
    unsigned long start = millis();
    while (millis() - start < timeout_ms) {
        while (gsmSerial.available()) {
            char c = (char)gsmSerial.read(); resp += c; Serial.write(c);
            if (resp.indexOf("OK") >= 0) return true;
            if (resp.indexOf("ERROR") >= 0) return false;
        }
    }
```

```
    return false;
}

void setup() {
    Serial.begin(PC_BAUD); delay(500);
    Serial.println("\n[ESP32] Modem Baud Lock to 115200");
    pinMode(MODEM_POWERON_PIN, OUTPUT);  digitalWrite(MODEM_POWERON_PIN, HIGH);
    pinMode(MODEM_RST_PIN, OUTPUT);       digitalWrite(MODEM_RST_PIN, HIGH);
    pinMode(MODEM_PWRKEY_PIN, OUTPUT);
    digitalWrite(MODEM_PWRKEY_PIN, HIGH); delay(100);
    digitalWrite(MODEM_PWRKEY_PIN, LOW);  delay(1000);
    digitalWrite(MODEM_PWRKEY_PIN, HIGH);
    delay(10000);

    long workingBaud = 0;
    for (int i = 0; i < NUM_BAUDS; i++) {
        gsmSerial.begin(BAUDS[i], SERIAL_8N1, MODEM_RX_PIN, MODEM_TX_PIN);
        delay(500);
        if (sendAndWaitForOK("AT", 1500)) { workingBaud = BAUDS[i]; break; }
        gsmSerial.end();
    }
    if (!workingBaud) { Serial.println("FAILED"); return; }

    sendAndWaitForOK("AT+IPR=115200",   2000);
    sendAndWaitForOK("AT+IPREX=115200", 2000);   /* CRITICAL — survives power cycle */
    sendAndWaitForOK("AT&W",            2000);

    Serial.println("DONE. Power-cycle the modem.");
    gsmSerial.begin(115200, SERIAL_8N1, MODEM_RX_PIN, MODEM_TX_PIN);
}

void loop() {
    if (Serial.available())   gsmSerial.write((char)Serial.read());
    if (gsmSerial.available()) Serial.write((char)gsmSerial.read());
}
```

## 5.2 — ESP32 deep-sleep sketch (release UART pins to STM32)

Once the baud is locked, this second sketch powers on the modem and then puts the ESP32 into deep sleep. After this, the ESP32 stops driving GPIO 26/27 and the STM32 can use those pins to talk to the modem directly.

```
#include <esp_sleep.h>
#include <Arduino.h>

#define MODEM_PWRKEY_PIN  4
#define MODEM_POWERON_PIN 12
#define MODEM_RST_PIN     5
#define MODEM_RX_PIN      27
#define MODEM_TX_PIN      26

void setup() {
  Serial.begin(115200);
  Serial.println("\n[ESP32] Power-on then sleep");

  pinMode(MODEM_POWERON_PIN, OUTPUT); digitalWrite(MODEM_POWERON_PIN, HIGH);
  pinMode(MODEM_RST_PIN, OUTPUT);     digitalWrite(MODEM_RST_PIN, HIGH);
  pinMode(MODEM_PWRKEY_PIN, OUTPUT);
  digitalWrite(MODEM_PWRKEY_PIN, HIGH); delay(100);
  digitalWrite(MODEM_PWRKEY_PIN, LOW);  delay(1000);
  digitalWrite(MODEM_PWRKEY_PIN, HIGH);

  Serial.println("[ESP32] Modem powered on. Waiting 8 sec for boot...");
  delay(8000);

  /* Release UART pins so STM32 can drive them */
  pinMode(MODEM_RX_PIN, INPUT);
  pinMode(MODEM_TX_PIN, INPUT);

  Serial.println("[ESP32] Going to deep sleep — STM32 takes over");
  Serial.flush();
  esp_deep_sleep_start();
}
```

```
void loop() {} /* never reached */
```

Order of use: First flash the Factory Reset utility, run it once, then power-cycle. Then flash the deep-sleep sketch. Watch Serial Monitor until you see «Going to deep sleep», then close Arduino IDE. From this point on, never use Arduino IDE again — STM32 owns the modem.

# 6. STM32 Firmware (added to MicDAC project)

All additions live in the existing MicDAC project. The audio code from Phase 3 stays in place but is COMMENTED OUT for Phase 5.4 testing because TIM3+I2C blocking can hang the CPU when the MCP4725 is not in the circuit.

## 6.1 — Includes and defines

```
/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>
#include <stdbool.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PD */
#define BUF_SIZE      256
#define HALF_SIZE     (BUF_SIZE / 2)
#define MCP4725_ADDR  (0x60 << 1)

/* LTE config — UPDATE these to match the current ngrok session */
#define PHONE_NUMBER       "01000000001"
#define NGROK_HOST         "0.tcp.eu.ngrok.io"
#define NGROK_PORT         12930
#define APN                "internet.vodafone.net"

#define LTE_RX_BUF_SIZE    512
/* USER CODE END PD */
```

## 6.2 — Variables and prototypes

```
/* USER CODE BEGIN PV */
uint16_t adc_buffer[BUF_SIZE];
volatile uint8_t  half_ready = 0;
volatile uint8_t  full_ready = 0;
volatile uint16_t dac_play_idx = 0;

volatile uint8_t  lte_rx_byte;            /* 1-byte slot for UART RX IT */
volatile char     lte_rx_buf[LTE_RX_BUF_SIZE];
volatile uint16_t lte_rx_idx = 0;
volatile uint8_t  lte_registered = 0;
/* USER CODE END PV */

/* USER CODE BEGIN PFP */
void lte_clear_rx(void);
bool lte_wait_for(const char* expected, uint32_t timeout_ms);
bool lte_send_at(const char* cmd, const char* expected, uint32_t timeout_ms);
void lte_bring_up(void);
/* USER CODE END PFP */
```

## 6.3 — main() initialization (USER CODE BEGIN 2)

```
/* USER CODE BEGIN 2 */
/* Audio disabled during Phase 5.4 LTE testing.  Re-enable in Phase 6
   when MCP4725 is back in the circuit. */
// HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, BUF_SIZE);
// HAL_TIM_Base_Start(&htim2);
// HAL_TIM_Base_Start_IT(&htim3);

/* ESP32 already powered on the modem. Now do AT bring-up + REG to relay. */
lte_bring_up();
/* USER CODE END 2 */
```

## 6.4 — Main while-loop (LED status)

```
/* USER CODE BEGIN WHILE */
while (1)
{
    static uint32_t last_led_toggle = 0;

    if (lte_registered) {
        /* Slow blink at 1 Hz — registered idle */
```

```
        if (HAL_GetTick() - last_led_toggle >= 500) {
            HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
            last_led_toggle = HAL_GetTick();
        }
    } else {
        /* Not registered — LED off */
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
    }
}
/* USER CODE END WHILE */
```

## 6.5 — Helper functions (USER CODE BEGIN 4)

```
/* Clear the LTE RX buffer */
void lte_clear_rx(void) {
    __disable_irq();
    lte_rx_idx = 0;
    lte_rx_buf[0] = 0;
    __enable_irq();
}

/* Wait for substring in RX buffer, or timeout */
bool lte_wait_for(const char* expected, uint32_t timeout_ms) {
    uint32_t start = HAL_GetTick();
    while ((HAL_GetTick() - start) < timeout_ms) {
        if (strstr((const char*)lte_rx_buf, expected) != NULL) return true;
        HAL_Delay(10);
    }
    return false;
}

/* Send AT command and wait for expected response */
bool lte_send_at(const char* cmd, const char* expected, uint32_t timeout_ms) {
    lte_clear_rx();
    HAL_UART_Transmit(&huart1, (uint8_t*)cmd, strlen(cmd), 1000);
    HAL_UART_Transmit(&huart1, (uint8_t*)"\r\n", 2, 1000);
    return lte_wait_for(expected, timeout_ms);
}

/* UART RX interrupt callback — append byte to buffer */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART1) {
        if (lte_rx_idx < LTE_RX_BUF_SIZE - 1) {
            lte_rx_buf[lte_rx_idx++] = (char)lte_rx_byte;
            lte_rx_buf[lte_rx_idx]   = 0;
        } else {
            lte_rx_idx = 0;
            lte_rx_buf[0] = 0;
        }
        HAL_UART_Receive_IT(&huart1, (uint8_t*)&lte_rx_byte, 1);
    }
}
```

## 6.6 — The bring-up sequence

This is the heart of Phase 5.4. It runs once at boot and either succeeds (lte_registered = 1) or fails silently (registered stays 0). Each successful step blinks the LED N times for visual debugging.

```
static void debug_blink(uint8_t n) {
    for (uint8_t i = 0; i < n; i++) {
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);  /* ON */
        HAL_Delay(150);
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);    /* OFF */
        HAL_Delay(150);
    }
    HAL_Delay(500);
}

void lte_bring_up(void) {
    char cmd[128];

    /* Step 0: boot indicator */
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
    HAL_Delay(2000);
```

```
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);

    /* Start UART RX */
    HAL_UART_Receive_IT(&huart1, (uint8_t*)&lte_rx_byte, 1);

    /* ESP32 already powered the modem; wait briefly */
    HAL_Delay(2000);

    if (!lte_send_at("AT", "OK", 2000)) return;            debug_blink(1);
    if (!lte_send_at("ATE0", "OK", 2000)) return;          debug_blink(2);
    if (!lte_send_at("AT+CPIN?", "READY", 5000)) return;   debug_blink(3);
    if (!lte_send_at("AT+CSQ", "OK", 2000)) return;        debug_blink(4);

    snprintf(cmd, sizeof cmd, "AT+CGDCONT=1,\"IP\",\"%s\"", APN);
    if (!lte_send_at(cmd, "OK", 2000)) return;             debug_blink(5);

    if (!lte_send_at("AT+CGACT=1,1", "OK", 10000)) return; debug_blink(6);
    lte_send_at("AT+NETOPEN", "OK", 5000);                 debug_blink(7);

    snprintf(cmd, sizeof cmd, "AT+CIPOPEN=0,\"TCP\",\"%s\",%d", NGROK_HOST, NGROK_PORT);
    if (!lte_send_at(cmd, "+CIPOPEN: 0,0", 15000)) return; debug_blink(8);

    char reg_payload[32];
    int  reg_len = snprintf(reg_payload, sizeof reg_payload, "REG %s\r\n", PHONE_NUMBER);

    snprintf(cmd, sizeof cmd, "AT+CIPSEND=0,%d", reg_len);
    if (!lte_send_at(cmd, ">", 3000)) return;              debug_blink(9);

    lte_clear_rx();
    HAL_UART_Transmit(&huart1, (uint8_t*)reg_payload, reg_len, 1000);

    if (lte_wait_for("OK", 5000)) {
        lte_registered = 1;
        debug_blink(10);   /* SUCCESS */
    }
}
```

# 7. Operating Procedure

## 7.1 — One-time setup (per device)

- Wire up everything per Section 3 (6 STM32-LilyGO wires + 4 button wires).
- Power both boards (STM32 on USB-C to PC, LilyGO on USB-C to phone charger).
- Connect LilyGO USB-C briefly to PC for ESP32 programming.
- Open Arduino IDE. Upload the Factory Reset Utility (Section 5.1). Wait for «DONE».
- Power-cycle the LilyGO (unplug USB-C, wait 10s, plug back in).
- Upload the ESP32 deep-sleep sketch (Section 5.2). Wait for «Going to deep sleep».
- Disconnect LilyGO from PC. Plug it back into the phone charger.
- Re-attach STM32 wires to LilyGO if you removed them.
- Build and flash the STM32 firmware in VS Code.

## 7.2 — Normal startup (every boot)

- Start relay.py on laptop.
- Start ngrok: ngrok tcp 5555. Note the new public address.
- Update NGROK_HOST and NGROK_PORT in main.c if changed.
- Build & flash STM32.
- Power on the LilyGO board (USB-C charger).
- Power on the STM32 (USB-C to PC).
- Wait ~25 seconds for the LED to start slow-blinking.

## 7.3 — LED state legend

| LED state | Meaning |
|---|---|
| Off (right after reset) | Code starting |
| Solid ON for 2 seconds | Boot indicator (start of lte_bring_up) |
| 1 quick blink | AT command OK |
| 2 quick blinks | ATE0 OK (echo off) |
| 3 quick blinks | SIM card READY |
| 4 quick blinks | Signal strength check OK |
| 5 quick blinks | APN configured |
| 6 quick blinks | Data session activated |
| 7 quick blinks | IP stack opened (NETOPEN) |
| 8 quick blinks | TCP socket to relay opened |
| 9 quick blinks | Got > prompt for sending REG |
| 10 quick blinks | ✓ REGISTERED with relay (success!) |
| Slow 1 Hz blink (forever) | Registered, idle, ready for incoming calls |
| LED stops at N blinks | Step N+1 failed — see Section 8 |

# 8. Lessons Learned (the hard way)

Phase 5.4 was the hardest phase by far. Seven distinct issues were hit and solved over the debugging session. Each is documented here so the same time isn't wasted next time.

## 8.1 — Modem silent garbled output at 115200

Symptom: Arduino bridge sketch printed unreadable bytes from the modem.
Cause: SIMCom A7608 default baud is not always 115200. The second LilyGO board's modem shipped at a different rate (likely 9600 or 921600).
Fix: Auto-baud detect by trying multiple rates, then permanently lock with both AT+IPR=115200 AND AT+IPREX=115200 followed by AT&W. Without IPREX, the modem reverts on next power-cycle.

## 8.2 — TIM3 ISR hung the CPU

Symptom: After flashing the LTE firmware, the LED stayed solid ON forever; nothing else ran.
Cause: TIM3 ISR fires every 125µs and calls HAL_I2C_Master_Transmit to the MCP4725. With the MCP4725 NOT in the circuit (Phase 5.4 only tests LTE), the I²C transmit blocks until timeout. The blocking call uses HAL_GetTick() which depends on SysTick — but SysTick is at lower priority than TIM3, so it can't fire to advance the timeout. CPU hangs forever.
Fix: Comment out HAL_TIM_Base_Start_IT(&htim3) during Phase 5.4 testing. Re-enable in Phase 6 when audio + LTE are integrated and the MCP4725 is connected.

## 8.3 — STM32 PA8 alone did not reliably hold ESP32 in reset

Symptom: Even with PA8 driven LOW, the ESP32 was still partially active and seemed to be driving the UART pins, conflicting with STM32.
Cause: The LilyGO board's CH9102 USB-serial chip has its own DTR/RTS connections to the ESP32's EN pin. When USB-C is connected (even just for power), CH9102 can interfere.
Fix: Don't rely solely on PA8. Flash the ESP32 with a one-time «power-on then deep-sleep» sketch. After this, the ESP32 is in true low-power state with all pins released.

## 8.4 — PuTTY was not sending \r\n on Enter

Symptom: REG command sent the right bytes, but the relay never recognized a complete line.
Cause: PuTTY by default sends only carriage return (\r) on Enter. The relay's readline() waits for newline (\n).
Fix: Use Arduino IDE Serial Monitor with line ending set to «Both NL & CR». It always sends \r\n correctly.

## 8.5 — STM32 firmware byte count mismatch for REG

Symptom: AT+CIPSEND=0,18 followed by «REG 01000000001» produced \+CIPSEND: 0,18,18 but relay never registered.
Cause: «REG 01000000001» is 15 chars; with \r\n that's 17 bytes, not 18. Sending CIPSEND=0,18 made the modem buffer 18 bytes including a stray byte that broke the line.
Fix: Compute byte count carefully: 3 (REG) + 1 (space) + 11 (phone) + 2 (\r\n) = 17.

## 8.6 — ngrok free tier requires credit-card verification for TCP

Symptom: ngrok tcp 5555 errored with «You must add a credit or debit card».
Cause: ngrok recently restricted TCP tunnels to verified accounts to prevent abuse.
Fix: Add a card on the ngrok dashboard. ngrok performs a $0 verification charge only — does not actually charge. For long-term hosting, switch to a paid VPS (~$4/month) where the address never changes.

## 8.7 — ngrok address changes per session (free tier)

Symptom: After restarting ngrok, the public address (host + port) is different.
Cause: Free tier dynamic addresses.

Fix for development: Update NGROK_HOST and NGROK_PORT in STM32 firmware before each test session. For demo day: start ngrok, note address, flash STM32 with that address, demo within the session.

# 9. What's Next

Phase 5.4 is the foundation for everything in Submission 1. Now that one device can register with the relay automatically, the remaining work is:

| Phase | Description | Time |
|---|---|---|
| 5.5 | Build Device 2 — same hardware as Device 1, but PHONE_NUMBER = "01000000002". Run through Sections 5.1, 5.2, 7.1, 7.2 again on the second LilyGO + STM32. | ~2 hrs |
| 6.1 | Add CALL / INC / ANS / GO / HUP state machine to STM32 firmware. User pushes Call → STM sends "CALL ". Other side gets "INC ". User pushes Answer → STM sends "ANS". Both end up in TALK state. | ~3 hrs |
| 6.2 | Stream audio bytes between devices: re-enable TIM3, mic samples → packed bytes → CIPSEND → received bytes → DAC playback buffer. Both directions simultaneously. | ~6 hrs |
| 8 | Demo polish: status LED states for ringing/in-call, debounced button reading, video backup, latency measurement. | ~3 hrs |

After Phase 8, Submission 1 is demonstrated. Submission 2 (encryption with RSA + AES) gets layered on top later.

End of Phase 5.4 documentation.